

**MANUAL TÉCNICO Y DEL PROGRAMADOR:
ARQUITECTURA, MODELO DE DATOS Y
EXTENSIBILIDAD DEL SISTEMA JINHWA
CORPORATION**

*Documentación de Ingeniería de Software para el Sistema de Gestión del Club de
Taekwondo*

Corporación Deportiva Jinhwa Corporation

Departamento de Desarrollo e Ingeniería de Software

Framework Base: Patrón MVC Nativo en PHP 8.x

Agosto de 2026

Tabla de Contenidos

A continuación se detallan los módulos y directrices técnicas desarrolladas en el manual del programador:

1. Visión General y Pila Tecnológica	3
2. Configuración del Entorno de Desarrollo	3
3. Estructura y Árbol de Directorios	4
4. Arquitectura de Software y Ciclo de Vida HTTP	5
5. Capa Núcleo (Core Framework)	6
5.1 Enrutador Central (Router.php)	6
5.2 Controlador Base (Controller.php)	7
5.3 Modelo Base y Singleton de Conexión (Database / Model)	7
5.4 Motor de Seguridad y Sesiones (Security.php)	8
6. Esquema de Base de Datos Normalizado	9
6.1 Modelo Relacional y Entidades Principales	9
6.2 Integridad Referencial y Reglas de Cascada	11
7. Guía Paso a Paso para Nuevos Módulos	11
8. Estándares y Convenciones de Codificación	13
9. Diagnóstico y Depuración Técnica	14
Referencias	15

1. Visión General y Pila Tecnológica

El sistema Jinhwa Corporation ha sido edificado bajo los principios del paradigma de programación orientada a objetos (POO) y el patrón arquitectónico Modelo-Vista-Controlador (MVC), prescindiendo deliberadamente de frameworks de terceros pesados. Esta decisión técnica maximiza el rendimiento de ejecución, garantiza total soberanía sobre la lógica del código y reduce al mínimo la superficie de vulnerabilidades externas.

Tabla 1

Especificación de la Pila Tecnológica del Sistema

Capa / Dominio	Tecnología Principal	Función y Alcance
Servidor Web	Apache HTTP Server 2.4+	Reescritura de URLs mediante <code>mod_rewrite</code> y despacho Front Controller.
Backend Runtime	PHP 8.1+ (Nativo)	Ejecución de controladores, modelos, validaciones y autenticación RBAC.
Gestor de Base de Datos	MySQL 8.0+ / MariaDB 10.4+	Motor transaccional InnoDB con codificación <code>utf8mb4_unicode_ci</code> .
Frontend	HTML5, CSS3, JavaScript ES6	Plantillas modulares y renderizado del lado del servidor.
Entorno Local	Laragon (Windows)	Suite integrada de servicios para desarrollo y pruebas.

Nota. El aplicativo no requiere administradores de dependencias tipo *Composer* para su operación base.

2. Configuración del Entorno de Desarrollo

Para desplegar localmente el entorno de ingeniería se requiere seguir la secuencia de instalación:

- 1. Clonación del Repositorio:** Situar la raíz del proyecto en la carpeta pública del servidor web (por ejemplo, `c:\laragon\www\jinhwa`).
- 2. Aprovisionamiento de Base de Datos:**

```
CREATE DATABASE jinhwa_corporation CHARACTER SET utf8mb4 COLLATE  
USE jinhwa_corporation;  
SOURCE c:/laragon/www/jinwha/jinhwa_corporation.sql;
```

3. **Parametrización de Conexión:** Ajustar los atributos del archivo `conexion.php` (host, usuario, clave y nombre de esquema).
4. **Generación de Administrador Inicial:** Ejecutar transitoriamente el script `crear_administrador.php` para emitir la clave encriptada con Bcrypt.

3. Estructura y Árbol de Directorios

La organización modular del proyecto aísla responsabilidades funcionales, como se aprecia en la Figura 1.

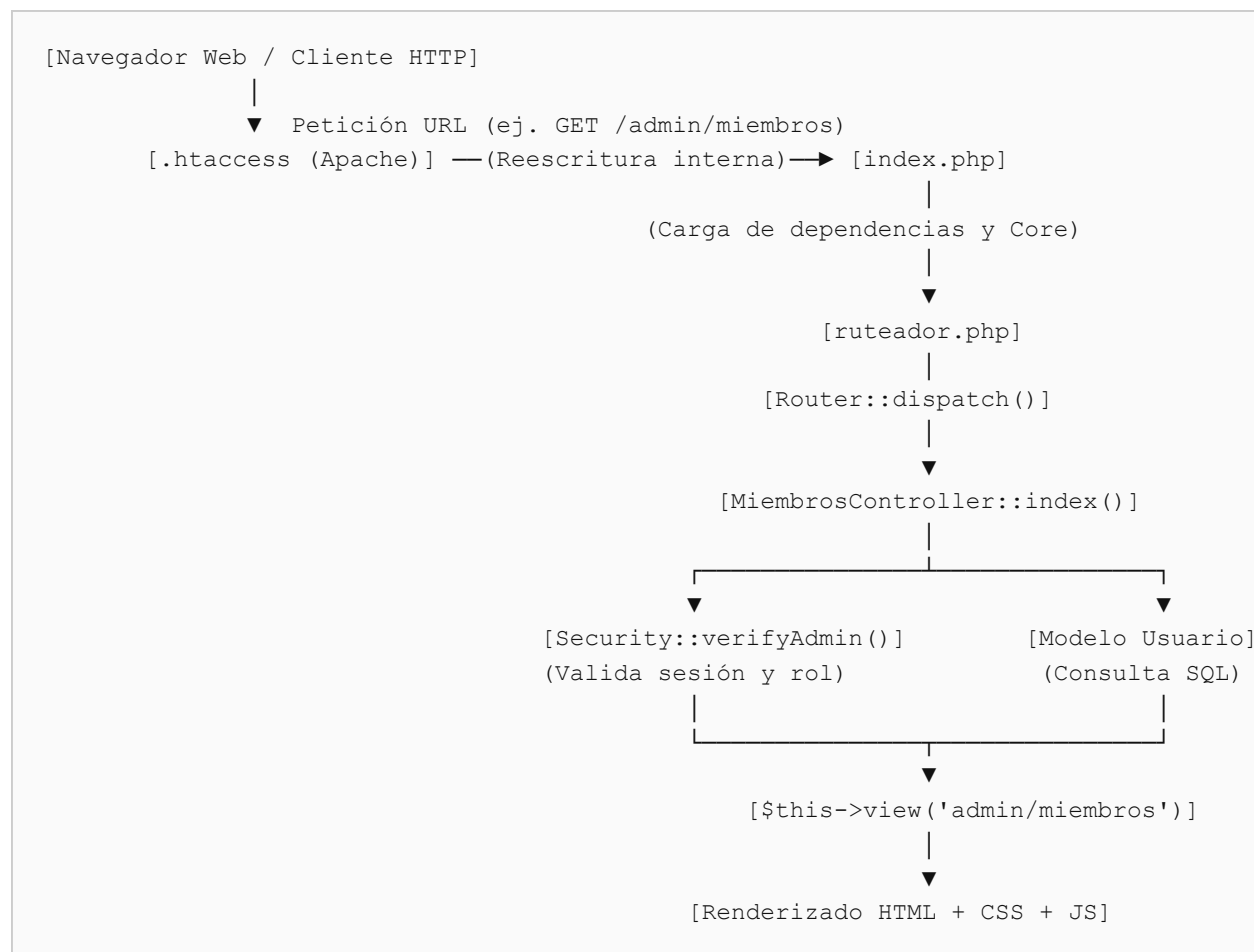
Figura 1*Árbol Estructural de Directorios y Archivos del Proyecto*

jinwha/	
├ .htaccess	# Reglas de reescritura Front Controller
├ index.php	# Punto de entrada unificado
├ ruteador.php	# Tabla de rutas y mapeo de controladores
├ conexion.php	# Clase Database (Patrón Singleton MySQLi)
├ core/	# Clases fundacionales del framework
│ └ Router.php	# Despachador HTTP REST/Clean URLs
│ └ Controller.php	# Clase base de vistas y redirecciones
│ └ Model.php	# Clase base con inyección de conexión
│ └ Roles.php	# Constantes de roles y evaluadores
│ └ Security.php	# Motor de 6 capas de protección de sesión
├ controladores/	# Lógica de aplicación por dominios
│ └ Autenticacion/	# Acceso, registro y finalización de perfil
│ └ Web/	# Portal público y páginas informativas
│ └ Administracion/	# Gestión de miembros, sedes y configuración
│ └ Maestro/	# Alumnos y solicitudes de examen
│ └ Estudiante/	# Contenido pedagógico e historial
│ └ Usuario/	# Módulos comunes (perfil, calendario)
├ modelos/	# Entidades con lógica SQL y transaccional
├ vistas/	# Plantillas PHP segmentadas por rol
│ └ layout/	# Cabeceras, barras de navegación y pies
│ └ web/, administracion/, maestro/, estudiante/, usuario/	
├ storage/	# Almacenamiento de archivos y multimedia
└ documentacion/	# Manuales técnicos y diagramas

Nota. Toda petición HTTP no estática es canalizada a través de `index.php`.

4. Arquitectura de Software y Ciclo de Vida HTTP

El procesamiento de una solicitud en el sistema Jinhwa Corporation sigue un flujo determinístico, documentado en la Figura 2.

Figura 2*Diagrama de Secuencia del Ciclo de Vida de una Petición HTTP*

Nota. El flujo garantiza que ningún controlador sea instanciado sin pasar por el Router y los filtros de seguridad.

5. Capa Núcleo (Core Framework)

El núcleo del aplicativo reside en el espacio de nombres App\Core y App\Config.

5.1 Enrutador Central (Router.php)

La clase App\Core\Router indexa las rutas admitidas para verbos GET y POST. Normaliza la ruta URI entrante suprimiendo el subdirectorio base de Laragon y despacha el método correspondiente:

```
// Ejemplo de registro en ruteador.php
$router->get('/admin/miembros', [MiembrosController::class, 'index']);
```

```
$router->post('/admin/miembros/create', [MiembrosController::class, 'store'])
```

5.2 Controlador Base (Controller.php)

Provee los métodos estándar para la renderización de vistas y saltos de redirección:

- `view(string $view, array $data = [])`: Aplica `extract($data)` sobre el vector asociativo e incluye la plantilla ubicada en `vistas/{$view}.php`.
- `redirect(string $url)`: Resuelve la ruta base del servidor y emite una cabecera HTTP Location seguida de `exit`.

5.3 Modelo Base y Singleton de Conexión (Database / Model)

La clase `Database` encapsula el driver `mysqli` bajo el patrón Singleton, impidiendo la apertura innecesaria de múltiples conexiones concurrentes:

```
namespace App\Core;
use App\Config\Database;

class Model {
    protected $db;
    public function __construct() {
        $this->db = Database::getInstance()->getConnection();
    }
}
```

5.4 Motor de Seguridad y Sesiones (Security.php)

La clase `App\Core\Security` implementa seis capas de protección para mitigar ataques de secuestro de sesión (Session Hijacking), fijación de sesión (Session Fixation) y accesos no autorizados:

Tabla 2*Capas de Protección de Seguridad en Security.php*

Capa	Mecanismo	Propósito y Comportamiento
1	Verificación de Identidad	Comprueba la existencia de <code>\$_SESSION['id']</code> . Si no existe, aborta a <code>/login?error=no_session</code> .
2	Timeout de Inactividad	Caduca la sesión si transcurren más de 1.800 segundos (30 min) desde la última interacción.
3	Huella User-Agent	Compara la cabecera <code>HTTP_USER_AGENT</code> original. Si difiere, destruye la sesión por intento de suplantación.
4	Regeneración de ID	Invoca <code>session_regenerate_id(true)</code> periódicamente cada 30 minutos para mitigar fijación.
5	Tiempo Límite Absoluto	Destruye forzosamente cualquier sesión que supere 28.800 segundos (8 horas continuas de uso).
6	Autorización por Rol / RBAC	Métodos <code>verifyAdmin()</code> , <code>verifyMaestro()</code> y <code>verifyPermission()</code> para control estricto.

Nota. Todos los intentos fallidos son registrados con timestamp en `debug_login.txt`.

6. Esquema de Base de Datos Normalizado

La base de datos se encuentra normalizada en Tercera Forma Normal (3FN), segregando la identidad civil (*personas*), la seguridad de acceso (*credenciales*) y los perfiles específicos de rol (*perfil_deportistas* y *perfil_maestros*).

6.1 Modelo Relacional y Entidades Principales

La Tabla 3 describe las tablas maestras del sistema y sus relaciones cardinales.

Tabla 3*Diccionario de Tablas Centrales de la Base de Datos*

Tabla	Llave Primaria y Foráneas	Atributos Principales
personas	id_persona (PK) id_sede (FK → sedes)	nombre, apellido, num_doc, tipo_documento, telefono, foto_perfil, activo, created_at.
credenciales	id_credencial (PK) id_persona (FK UNIQUE)	correo (UNIQUE), clave (hash Bcrypt), rol (ENUM), permisos_extra (JSON).
perfil_deportistas	id_perfil (PK) id_persona (FK UNIQUE) id_grado, id_categoria (FK)	fecha_n, peso, division, ctgc, eps, rh.
perfil_maestros	id_perfil (PK) id_persona (FK UNIQUE) id_grado (FK)	descripcion_perfil, logros, mostrar_en_web.
solicitudes_ascenso	id_solicitud (PK) id_deportista, id_maestro (FK) id_grado_actual, id_grado_solicitado (FK)	estado (pendiente, aprobado, rechazado), observaciones, fecha_solicitud, fecha_resolucion.
teorias	id_teoría (PK) id_grado, id_tipo_teoría (FK)	nombre, contenido, url_video.
sedes, grados, eventos	PKs autoincrementales	Catálogos maestros de sedes físicas, grados marciales y cronograma de eventos.

Nota. El esquema completo reside en el archivo *jinhwa_corporation.sql*.

6.2 Integridad Referencial y Reglas de Cascada

Las llaves foráneas ligadas a `personas.id_persona` están definidas con `ON DELETE CASCADE` y `ON UPDATE CASCADE`, garantizando que la supresión de un afiliado limpie automáticamente sus credenciales y su ficha técnica correspondiente.

7. Guía Paso a Paso para Nuevos Módulos

Para incorporar una nueva entidad al sistema (por ejemplo, el módulo de *Noticias*), el desarrollador debe seguir rigurosamente la secuencia de seis pasos:

Paso 1: Creación de la Tabla en Base de Datos

```
CREATE TABLE noticias (
    id_noticia INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(255) NOT NULL,
    contenido TEXT NOT NULL,
    publicado TINYINT(1) DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Paso 2: Implementación del Modelo (modelos/Noticia.php)

```
<?php
namespace App\Models;
use App\Core\Model;

class Noticia extends Model {
    public function getAll() {
        $result = $this->db->query("SELECT * FROM noticias ORDER BY created_at");
        return $result ? $result->fetch_all(MYSQLI_ASSOC) : [];
    }

    public function create($data) {
        $stmt = $this->db->prepare("INSERT INTO noticias (titulo, contenido)");
        $stmt->bind_param("ss", $data['titulo'], $data['contenido']);
        return $stmt->execute();
    }
}
```

Paso 3: Creación del Controlador (controladores/Administracion/NoticiasController.php)

```
<?php
namespace App\Controllers\Administracion;
use App\Core\Controller;
use App\Core\Security;
use App\Models\Noticia;

class NoticiasController extends Controller {
    public function index() {
        Security::verifySession();
        Security::verifyAdmin();
        $noticias = (new Noticia())->getAll();
        $this->view('administracion/noticias', ['noticias' => $noticias]);
    }
}
```

```
}
}
```

Paso 4: Creación de la Vista (vistas/administracion/noticias.php)

```
<?php require_once __DIR__ . '/../layout/header_admin.php'; ?>
<div class="container">
    <h1>Tablón de Noticias Institucionales</h1>
    <?php foreach ($noticias as $item): ?>
        <article>
            <h3><?= htmlspecialchars($item['titulo']) ?></h3>
            <p><?= nl2br(htmlspecialchars($item['contenido'])) ?></p>
        </article>
    <?php endforeach; ?>
</div>
<?php require_once __DIR__ . '/../layout/footer_admin.php'; ?>
```

Paso 5: Registro del Modelo en index.php

```
require_once __DIR__ . '/modelos/Noticia.php';
```

Paso 6: Registro de Rutas en ruteador.php

```
require_once __DIR__ . '/controladores/Administracion/NoticiasController.php';
use App\Controllers\Administracion\NoticiasController;

$route->get('/admin/noticias', [NoticiasController::class, 'index']);
```

8. Estándares y Convenciones de Codificación

Todos los desarrolladores deben ceñirse a las siguientes directrices:

- **Prepared Statements Obligatorios:** Toda consulta con parámetros externos debe usar `$this->db->prepare()` y `bind_param()`. Queda estrictamente prohibida la concatenación de variables en sentencias SQL.
- **Sanitización de Salidas:** Utilizar `htmlspecialchars()` en las vistas para mitigar ataques XSS.
- **Nomenclatura:** Clases en `PascalCase`, métodos y variables en `camelCase`, tablas y campos en `snake_case`, y URLs en `kebab-case`.

9. Diagnóstico y Depuración Técnica

Para la resolución de incidencias en producción o desarrollo:

- **Trazas de Autenticación:** Inspeccionar el archivo `debug_login.txt` en la raíz para auditar accesos exitosos o bloqueos de sesión.
- **Errores de Enrutamiento 404:** Confirmar que el módulo `mod_rewrite` de Apache esté activo y que el archivo `.htaccess` contenga la regla de Front Controller hacia `index.php`.
- **Fallo de Vistas:** Asegurar que el nombre de plantilla pasado a `$this->view('ruta/archivo')` no incluya la extensión `.php`.

Referencias

- American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). <https://doi.org/10.1037/0000165-000>
- Freeman, E., & Robson, E. (2020). *Head first design patterns: Building extensible and maintainable object-oriented software* (2nd ed.). O'Reilly Media.
- The PHP Group. (2024). *PHP manual: MySQL Improved extension (MySQLi)*. <https://www.php.net/manual/es/book.mysqli.php>